HW7

Nathan Krieger

2026-03-23

Problem 1: Multiple Linear Regression

Suppose you work for a consulting firm. Your manager gives you the Boston dataset (part of library ISLR2) and asks you to build a multiple linear regression model to predict median home prices (medv). In typical form, he gives you no instructions or input. The final deliverable should be a model that can accurately predict median home price from a set of predictors.

Create a report (no more than 1 page) that includes the following items/discussion:

```r
library(ISLR2)
library(leaps)


set.seed(123)

# part a
best_model <- regsubsets(medv ~ ., data = Boston, nbest = 1, nvmax = 13)

summary(best_model)
```

```
## Subset selection object
## Call: regsubsets.formula(medv ~ ., data = Boston, nbest = 1, nvmax = 13)
## 12 Variables  (and intercept)
##         Forced in Forced out
## crim        FALSE      FALSE
## zn          FALSE      FALSE
## indus       FALSE      FALSE
## chas        FALSE      FALSE
## nox         FALSE      FALSE
## rm          FALSE      FALSE
## age         FALSE      FALSE
## dis         FALSE      FALSE
## rad         FALSE      FALSE
## tax         FALSE      FALSE
## ptratio     FALSE      FALSE
## lstat       FALSE      FALSE
## 1 subsets of each size up to 12
## Selection Algorithm: exhaustive
##           crim zn  indus chas nox rm  age dis rad tax ptratio lstat
## 1  ( 1 )  " "  " " " "   " "  " " " " " " " " " " " " " "     "*"
## 2  ( 1 )  " "  " " " "   " "  " " "*" " " " " " " " " " " "     "*"
## 3  ( 1 )  " "  " " " "   " "  " " "*" " " " " " " " " " "*"     "*"
## 4  ( 1 )  " "  " " " "   " "  " " "*" " " "*" " " " " "*"     "*"
```

```
## 5  ( 1 )  " "  " " " "   " "  "*" "*" " " "*" " " " " "*"     "*"
## 6  ( 1 )  " "  " " " "   "*"  "*" "*" " " "*" " " " " "*"     "*"
## 7  ( 1 )  " "  "*" " "   "*"  "*" "*" " " "*" " " " " "*"     "*"
## 8  ( 1 )  "*"  "*" " "   "*"  "*" "*" " " "*" " " " " "*"     "*"
## 9  ( 1 )  "*"  "*" " "   " "  "*" "*" " " "*" "*" "*" "*"     "*"
## 10  ( 1 ) "*"  "*" " "   "*"  "*" "*" " " "*" "*" "*" "*"     "*"
## 11  ( 1 ) "*"  "*" " "   "*"  "*" "*" "*" "*" "*" "*" "*"     "*"
## 12  ( 1 ) "*"  "*" "*"   "*"  "*" "*" "*" "*" "*" "*" "*"     "*"
```

```r
# part b
smallest <- which.min(summary(best_model)$bic)

best <- coef(best_model, id = smallest)

# part c
reduced <- Boston[ , !(names(Boston) %in% c("rad", "tax"))]
regfit_reduced <- regsubsets(medv ~ ., data = reduced, nbest = 1, nvmax = 11)
# After checking assumptions
final_fit <- lm(medv ~ crim + zn + chas + nox + poly(rm, degree = 2) + dis + rad + tax + ptratio + poly


# part d
final_fit <- lm(medv ~ crim + zn + chas + nox + poly(rm, degree = 2) + dis + ptratio + poly(lstat, degre
```

(a)

Report your final model and its regression coefficients in a nicely summarized table.

```r
smallest <- which.min(summary(regfit_reduced)$bic)

best <- coef(regfit_reduced, id = smallest)
knitr::kable(best)
```

	x
(Intercept)	34.9998143
crim	-0.0804215
zn	0.0410901
chas	3.1704528
nox	-17.7187750
rm	4.0099787
dis	-1.4501099
ptratio	-0.8561764
lstat	-0.5477683

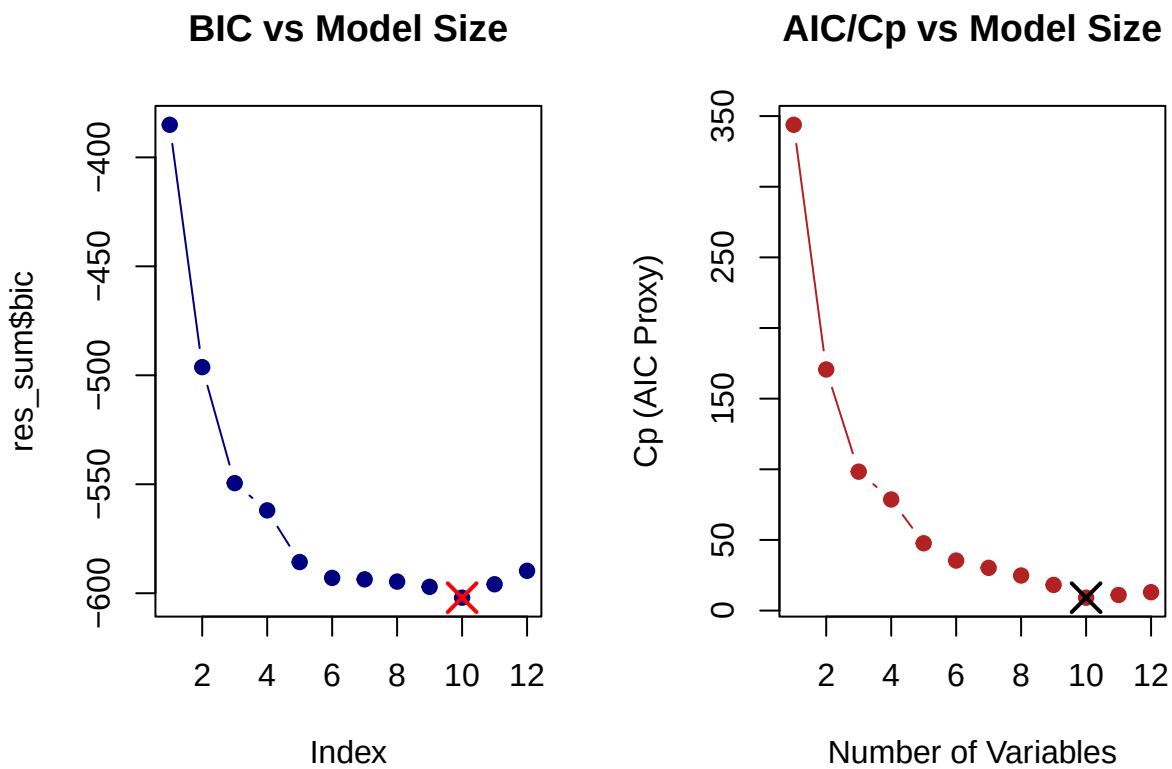
(b)

Not all predictors need to be in the model. What technique will you use to determine which predictors to include in the model and which to omit? Explain your workflow and justify how you decided on your "final" model. Directly address why you choose your final model over other candidate models.

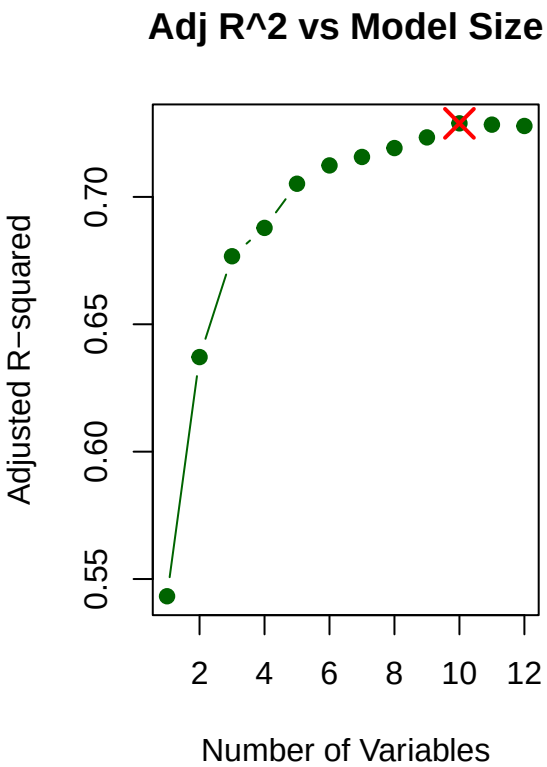
To find which predictors should be left out of the model, I used best subset selection.

```r
res_sum <- summary(best_model)
best_aic <- which.min(res_sum$cp)
best_r2  <- which.max(res_sum$adjr2)
```

```r
opt_size <- which.min(res_sum$bic)

par(mfrow = c(1, 2))
plot(res_sum$bic, type = "b", pch = 19, col = "navy", main = "BIC vs Model Size")
points(opt_size, res_sum$bic[opt_size], col = "red", cex = 2, pch = 4, lwd = 2)

plot(res_sum$cp, type = "b", pch = 19, col = "firebrick",
     xlab = "Number of Variables", ylab = "Cp (AIC Proxy)",
     main = "AIC/Cp vs Model Size")
points(best_aic, res_sum$cp[best_aic], col = "black", cex = 2, pch = 4, lwd = 2)
```




```r
plot(res_sum$adjr2, type = "b", pch = 19, col = "darkgreen",
     xlab = "Number of Variables", ylab = "Adjusted R-squared",
     main = "Adj R^2 vs Model Size")
points(best_r2, res_sum$adjr2[best_r2], col = "red", cex = 2, pch = 4, lwd = 2)
```



As the plots above display, 10 predictors is the best size for this model. The code itself handles the selection of this model (using regsubsets). To determine which predictors to include in the final model, I used Best Subset Selection. This is an exhaustive search method that evaluates every possible combination of the 13 available predictors. I selected the model that minimized the BIC. In the problem below, I check if the assumptions we made are true.

(c)

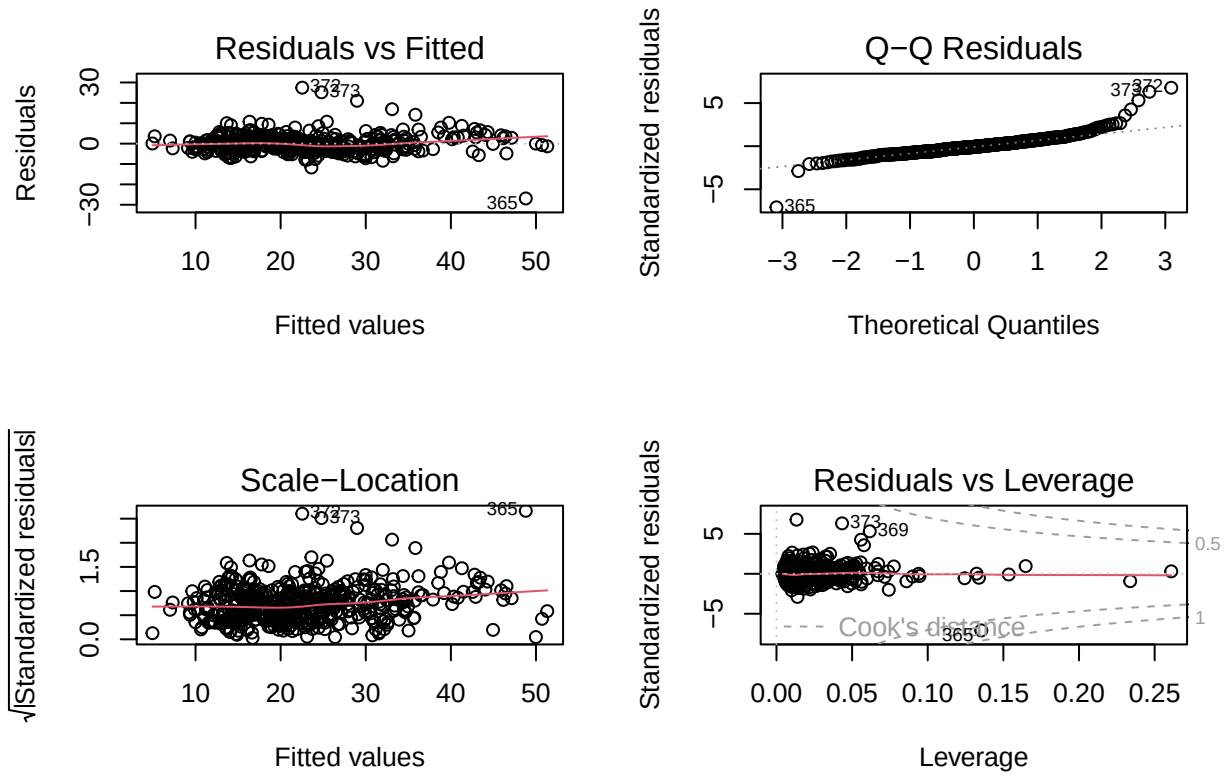
What assumptions were needed to obtain this final model? Are there any diagnostic checks you made to ensure those assumptions were valid? Did you update your final model to accommodate any assumption violations?

Linearity Independence Constant variance Normality of errors

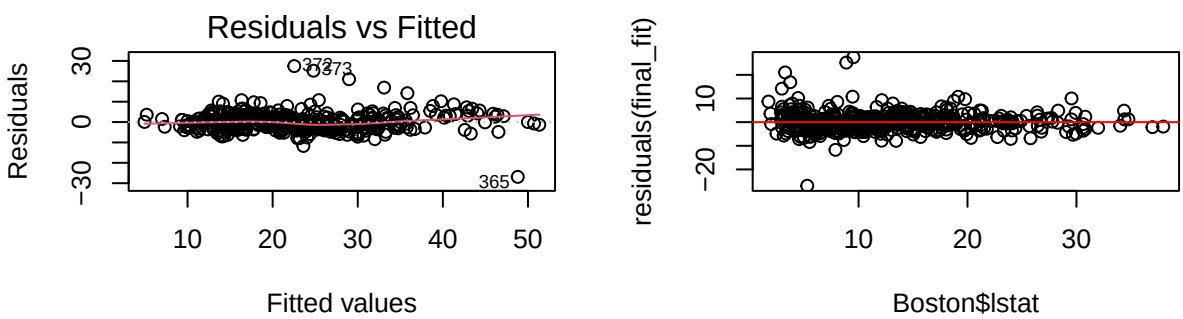
```
library(car)
```

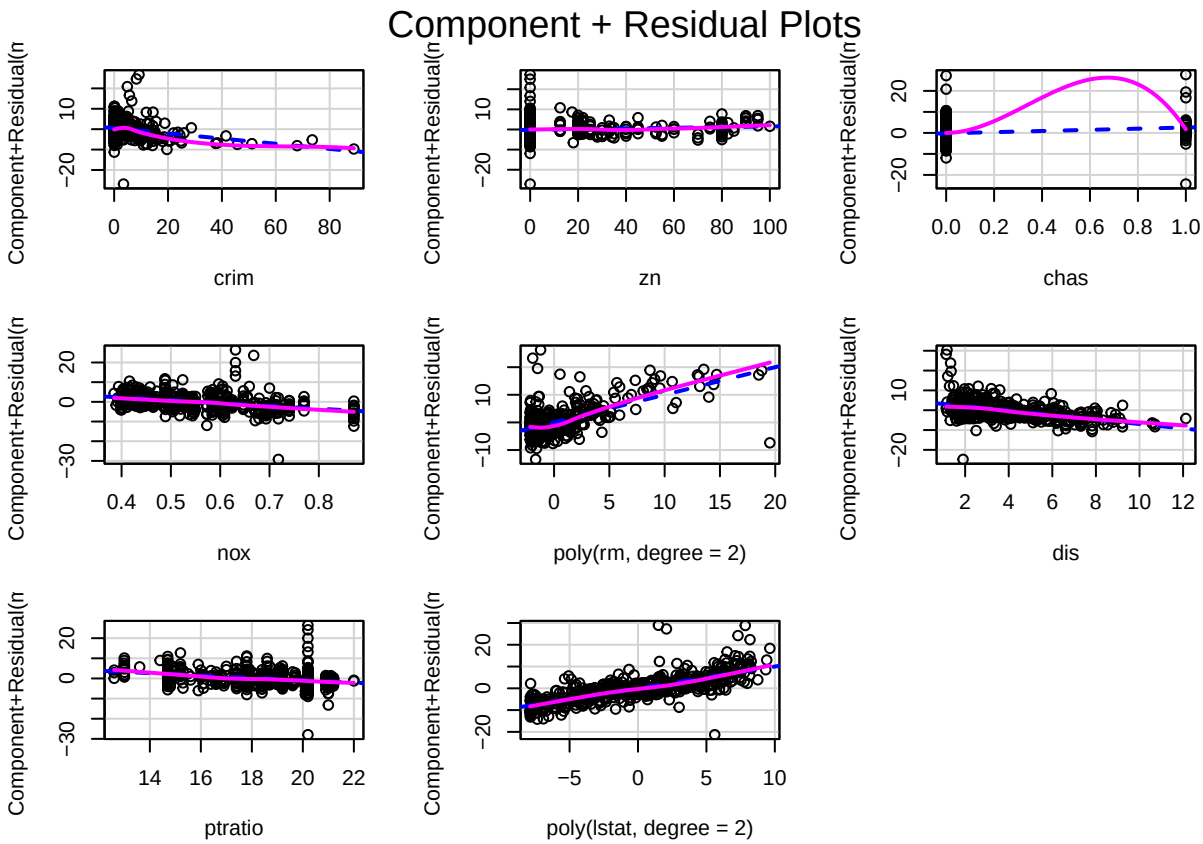
```
## Loading required package: carData
```

```
# Linearity

par(mfrow = c(2, 2))
plot(final_fit)
```




```
plot(final_fit, which = 1)

plot(Boston$lstat, residuals(final_fit))
abline(h = 0, col = "red")

crPlots(final_fit)
```





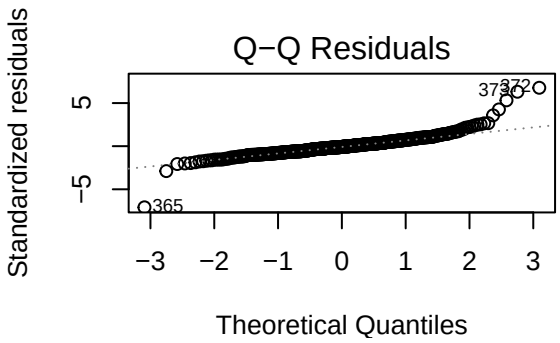

```r
# Independence

# plot(final_fit)

# Constant variance
library(lmtest)
```

```
## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
bptest(final_fit)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  final_fit
## BP = 60.725, df = 10, p-value = 2.642e-09
```

```r
# Normality of errors

plot(final_fit, which = 2)
```



Based on the plots above, I applied poly to rm and lstat in the final model. This is because their charts appeared curved. To check for independence I used the plot(final_fit). To check for constant variance I used the bptest method. I found that the variance is sufficiently constant based on the low p-value. I checked for normality of errors by plotting plot(final_fit, which = 2) and observed a roughly straight line, so no adjustment was needed.

(d)

Were there any potential issues related to your final model that you explored? Did you address these issues once identified?

```r
vif(final_fit)
```

```
##                              GVIF Df GVIF^(1/(2*Df))
## crim                     1.441221  1        1.200508
## zn                       2.187858  1        1.479141
## chas                     1.050692  1        1.025033
## nox                      3.040834  1        1.743799
## poly(rm, degree = 2)     2.400292  2        1.244704
## dis                      3.486821  1        1.867303
## ptratio                  1.440240  1        1.200100
## poly(lstat, degree = 2) 3.686169  2        1.385619
```

Yes. Based on the VIF method, I removed the rad and tax from the final model due to their high multicollinearity.

(e)

Can you obtain the bias and variance of this model? If so, explain how. If not, explain why not.

Since you are the only data scientist on the team, your manager reminds you to make every effort to produce a report that is rigorous but understandable to non-statisticians. Limita- tions/strengths of your pipeline to build this model should be discussed. Your workflow should be thoughtfully justified using the concepts/ideas we have discussed in class.

To obtain the bias, you need the true relationship (f(x)) which we do not have. To obtain variance the same is true.

Our model likely has some bias because we assumed a mostly linear relationship. If the true relationship is curved, the model is inherently biased.

Because we used BIC to select a smaller subset of variables, the variance of the model is lower compared to using all 13 predictors. This means our price predictions will be more stable if applied to a different suburb of Boston.

Problem 2: Follow-up to in-class activity

Let us recap what we learned from our in-class activity. See the post on Ed Discussion if you need a refresher.

(a)

Clearly the models in Part 2 and Part 3 are not equivalent. Explain again in plain language (using plots if you deem it helpful) why these two approaches will not give the same results.

Running one model with just a gender dummy variable assumes only the intercept is different while the slopes for other variables stay the same. Running two separate models assumes that all coefficients are different.

(b)

Load in the Insurance dataset again. Show how you can modify the model in Part 2 so that it is exactly equivalent to the models in Part 3. Present your code and results to show that the fitted models for males only and females only are exactly the same in Part 2 and Part 3 (after making your modification). Hint: use interaction terms.

```r
insurance = read.csv('~/git/DS301/HW7/insurance.csv', header = TRUE)

fit <- lm(charges ~ age * gender + bmi * gender, data = insurance)

summary(fit)
```

```
##
## Call:
## lm(formula = charges ~ age * gender + bmi * gender, data = insurance)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -16895  -6869  -4933   6872  47424
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -4515.219   2489.105  -1.814  0.06990 .
## age              246.919     31.612   7.811 1.14e-14 ***
## gendermale     -3497.572   3485.288  -1.004  0.31579
## bmi              241.319     73.483   3.284  0.00105 **
## age:gendermale    -8.287     44.544  -0.186  0.85245
## gendermale:bmi   168.550    102.742   1.641  0.10113
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11370 on 1332 degrees of freedom
## Multiple R-squared:  0.122,  Adjusted R-squared:  0.1187
## F-statistic: 37.03 on 5 and 1332 DF,  p-value: < 2.2e-16
```

```r
coef(fit)
```

```
##    (Intercept)            age     gendermale            bmi age:gendermale
##   -4515.218637     246.919325   -3497.572169     241.319332      -8.286575
## gendermale:bmi
##     168.549795
```

(c)

Other than domain knowledge, how might you determine using a data-driven way that an interaction term is needed in the model? Justify and then implement your approach here on the Insurance dataset.

We can do a partial F-Test. We compare a Reduced Model to an Augmented Model. If the improvement in the Sum of Squared Errors is statistically significant, the interaction is justified.

```
red <- lm(charges ~ age + bmi + gender, data = insurance)
full <- lm(charges ~ age*gender + bmi*gender, data = insurance)


anova(red, full)
```

```
## Analysis of Variance Table
##
## Model 1: charges ~ age + bmi + gender
## Model 2: charges ~ age * gender + bmi * gender
##   Res.Df        RSS Df Sum of Sq      F Pr(>F)
## 1   1334 1.7249e+11
## 2   1332 1.7215e+11  2 347824699 1.3457 0.2607
```

Based on my analysis. The p-value of the full model is 0.2607. This is much higher than a = 0.05 and therefore it is not worth the added complexity.

Problem 3: Predictions in the presence of multicollinearity (practice)

(a)

Is multicollinearity a problem for making accurate predictions? Make an educated guess based on what we have learned in class.

I would say no. It makes it hard to determine the effect of individual variables, but the model remains accurate.

(b)

Let us carry out a simulation study to answer this. We will simulate data with and without multicollinearity. This is our true model: Yi = B0 + B1Xi1 + B2Xi2 + epsilon_i,

where i = 1, . . . , 100, epsilon ~ N (u = 0, sigma2 = 4), B0 = 3, B1 = 2, and B2 = 4.

To generate your predictors with multicollinearity, use the following code:

```
set.seed(42)
x1 = runif(100)
x2 = 0.8*x1 + rnorm(100,0,0.1)

cor(x1, x2)
```

```
## [1] 0.9404249
```

Using these predictors, generate your Y values in R. Check the correlation between x1 and x2 using the cor() function. Report that value here.

Correlation is 0.9404249

(c)

Split your data into a training set and test set. Train your model on the training set.

```r
set.seed(42)

Y = 3 + 2*x1 + 4*x2 + rnorm(100, 0, 2)

df <- data.frame(Y, x1, x2)
head(df)
```

```
##           Y        x1        x2
## 1 10.627678 0.9148060 0.7640374
## 2  6.429860 0.9370754 0.6712764
## 3  5.844473 0.2861395 0.3864844
## 4  8.841213 0.8304476 0.7286480
## 5  7.181518 0.6417455 0.5223725
## 6  5.597670 0.5190959 0.4429318
```

```r
n <- dim(df)[1]

train_index <- sample(1:n,n/2,rep=FALSE)
train <- df[train_index,]
test <- df[-train_index,]

m <- lm(Y ~ x1 + x2, data = train)

pred <- predict(m, newdata = test)
test_mse <- mean((pred - test$Y)^2)
test_mse
```

```
## [1] 4.683967
```

Report the test MSE for this model.

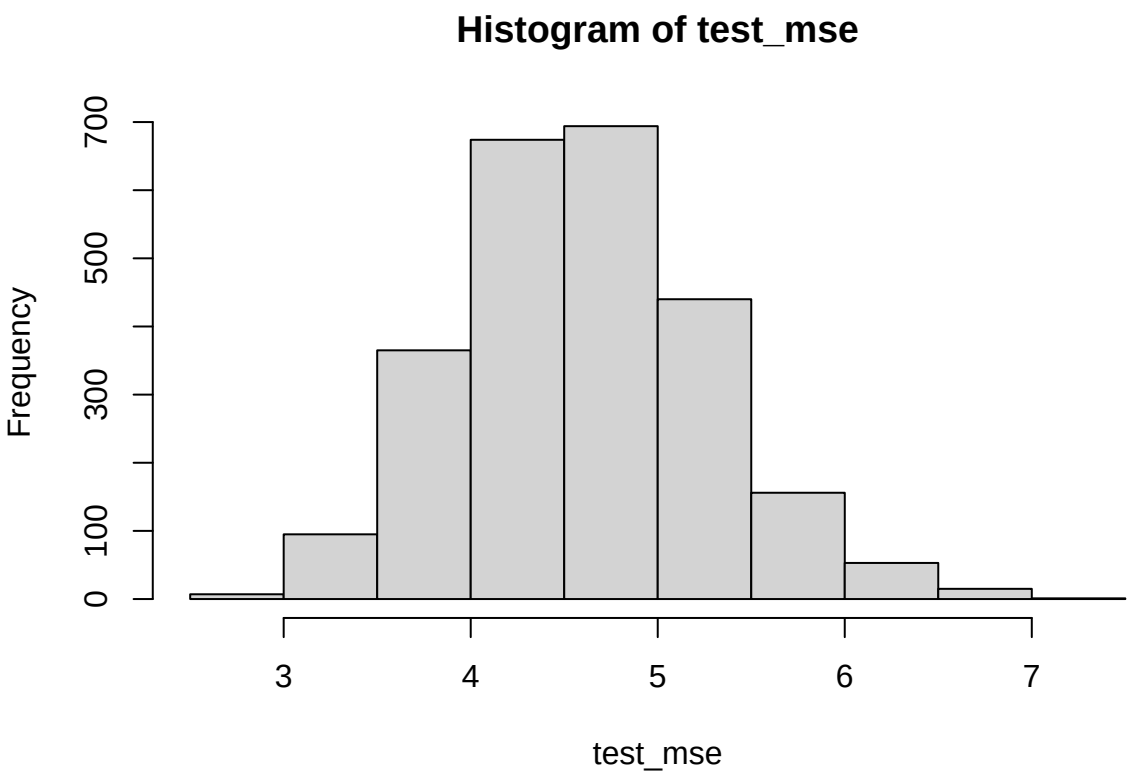
Test MSE: 4.683967

(d)

Repeat this process 2,500 times. Each time you will need to generate a random set of Y 's, fit a model on your training set, and then obtain the test MSE for that model. We do not need to generate new predictor values. Remember to store the test MSE for each iteration and do not set a seed within the loop. What is the mean test MSE in this setting when the predictors are highly correlated? Plot a histogram of your 2,500 test MSEs and comment on what you observe.

```r
set.seed(42)

test_mse <- rep(NA, 2500)
test_mat <- model.matrix(Y ~ ., data = test)

for(i in 1:2500) {
  # New random noise for Y
  y_sim <- 3 + 2*x1 + 4*x2 + rnorm(n, 0, 2)

  df <- data.frame(Y, x1, x2)

  train_index <- sample(1:n,n/2,rep=FALSE)
  train <- df[train_index,]
  test <- df[-train_index,]
```

```r
  # Fit and Predict
  m <- lm(Y ~ x1 + x2, data = train)
  pred <- predict(m, newdata = test)
  test_mse[i] <- mean((pred - test$Y)^2)
}

hist(test_mse)
```




```r
mean(test_mse)
```

```
## [1] 4.606293
```

Based on the plot we can see that the mean is around 4 which is around the sigma squared.

(e)

Now generate predictors without multicollinearity using the following code:

```r
set.seed(24)
x1 = runif(100)
x2 = rnorm(100,0,1)

cor(x1, x2)
```

```
## [1] 0.03316596
```

```r
Y = 3 + 2*x1 + 4*x2 + rnorm(100, 0, 2)
```

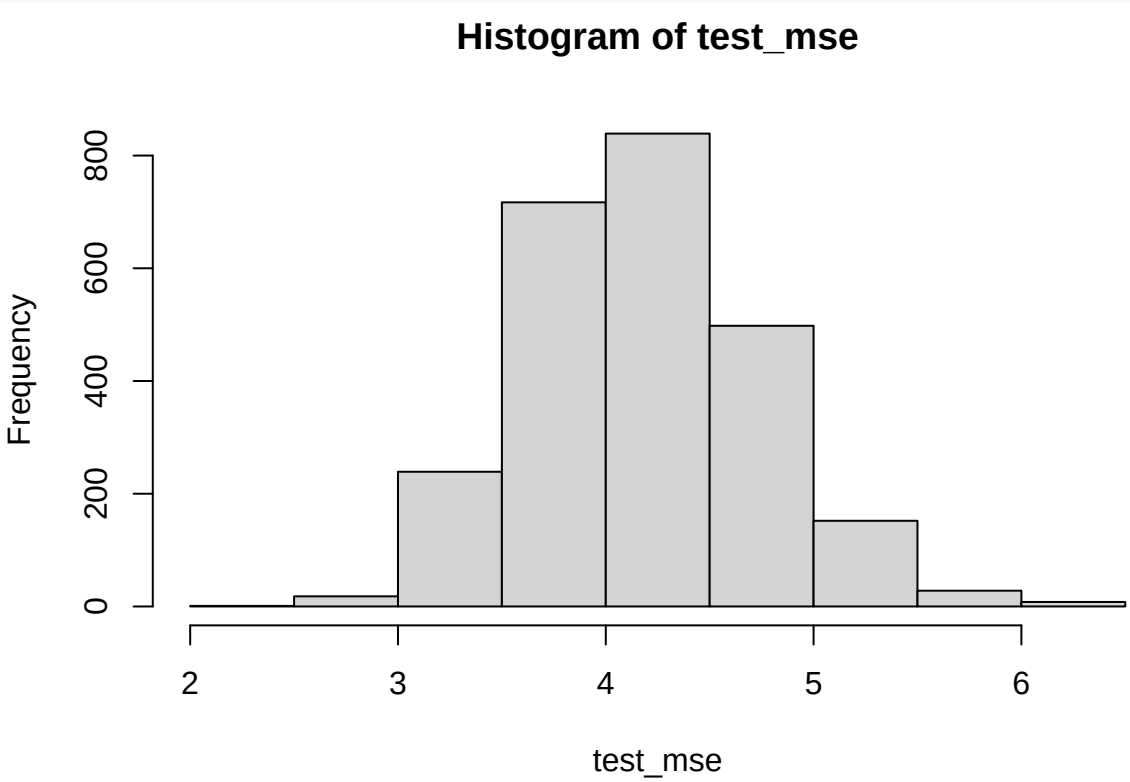
Using these predictors, generate your Y values in R. Check the correlation between x1 and x2. Report that value here.

Correlation: 0.03316596

(f)

Again run 2,500 simulations to obtain the test MSE of our model when the predictors are not correlated. What is the mean test MSE in this setting without multicollinearity? Plot a histogram of your 2,500 test MSEs and comment on what you see.

```r
set.seed(24)

test_mse <- rep(NA, 2500)
test_mat <- model.matrix(Y ~ ., data = test)

for(i in 1:2500) {
  # New random noise for Y
  y_sim <- 3 + 2*x1 + 4*x2 + rnorm(n, 0, 2)

  df <- data.frame(Y, x1, x2)

  train_index <- sample(1:n,n/2,rep=FALSE)
  train <- df[train_index,]
  test <- df[-train_index,]

  # Fit and Predict
  m <- lm(Y ~ x1 + x2, data = train)
  pred <- predict(m, newdata = test)
  test_mse[i] <- mean((pred - test$Y)^2)
}

hist(test_mse)
```



```r
mean(test_mse)
```

```
## [1] 4.181713
```

Based on the plot: the mean is the same (~4). This means that the correlation is not hurting the model.

(g)

Based on our simulation study, is multicollinearity a problem for making accurate predictions? Comment on your findings.

No. We can see both test MSEs are about the same. Multicollinearity increases variance but the variance cancels out. Both simulations have a mean test MSE around 4.